

ENTERPRISE TECHNICAL SPECIFICATION

WhatsApp API Gateway & AI Chat Support System

Architectural Deep Dive, Queue Concurrency Engine, Technology Trade-Off Analysis, Multi-Tier Anti-Ban Engine, and Complete REST / Webhook API Reference.

Architecture Style: Transactional Outbox + Async Event-Driven
Queue Technology: PostgreSQL (FOR UPDATE SKIP LOCKED)
Vector Storage: PostgreSQL + pgvector (768 Dimensions)
AI Engine: Google Gemini 1.5 Flash via LangChain JS
Protocol: Baileys Multi-Device WebSocket (Noise Protocol)
Documentation Version: v2.1.0 — Production Standard

1. System Architecture Overview

The WhatsApp Gateway is designed as a carrier-grade microservice that interfaces between internal enterprise applications (CRMs, ERPs, support bots) and real WhatsApp accounts. It handles three core responsibilities:

- **Persistent Session Lifecycle:** Links company WhatsApp numbers via multi-device QR code pairing, storing cryptographic Signal keys directly in PostgreSQL. Sockets survive container restarts without requiring re-scans.
- **Asynchronous Message Outbox:** Provides non-blocking REST endpoints returning `HTTP 202 Accepted`, scheduling outgoing messages through a strictly paced outbox queue.
- **Inbound Event Webhooks & RAG Support:** Receives customer messages, validates HMAC signatures, streams events to webhooks, and synthesizes grounded AI customer replies using Google Gemini 1.5 Flash and document vector context.

Architectural Decoupling Guarantee

Internal applications never interact directly with WhatsApp sockets or rate-limiting delays. They interact exclusively with synchronous REST endpoints. The internal outbox worker handles the pacing, network reconnects, and typing presence completely asynchronously.

2. Deep Dive: The Outbox Concurrency Engine

Why PostgreSQL `FOR UPDATE SKIP LOCKED` Was Chosen Over BullMQ (Redis) & RabbitMQ

Message queuing in high-reliability gateways presents unique requirements that differ from standard background job workers. Below is the technical rationale for why PostgreSQL's native row-level locking was selected:

Evaluation Dimension	PostgreSQL <code>SKIP LOCKED</code> (Chosen)	BullMQ (Redis)	RabbitMQ (AMQP)
Dual-Write Consistency	100% ACID Guaranteed Written in the same database transaction as the message record. Zero ghost messages.	High Risk Two-phase write over network. If Redis blinks after DB write, the job is lost forever.	High Risk Requires distributed transaction or dual-phase commit to prevent DB/broker desync.
Single-Stream per Number	Native SQL Filter <code>WHERE sessionId = :id ...</code> <code>LIMIT 1</code> guarantees strictly 1 message at a time per number.	Complex Requires BullMQ Pro grouped jobs or dynamic queue allocation per phone number.	Complex Requires single active consumer pattern per routing key or manual thread locking.
Multi-Hour Cooldown Scheduling	Zero Memory Cost Updates <code>nextRetryAt = timestamp</code> in DB. Sleeps on disk until exact millisecond.	Moderate Uses Redis sorted sets (ZSET). Retains entire payload in expensive RAM.	Poor Requires Dead-Letter Exchanges (DLX) and per-message TTL hackery.
Crash Recovery	WAL-Backed on Disk Survives power cuts or <code>kill -9</code>. Stale locks recovered automatically.	RAM Eviction Risk Default Redis eviction can drop jobs under memory pressure.	Durable Durable queues survive if explicitly configured, but recovery is heavy.
Operational Footprint	0 Extra Infrastructure Reuses existing PostgreSQL cluster.	Requires dedicated Redis cluster and Sentinel.	Requires Erlang VM, RabbitMQ cluster, and monitoring.

The "Dual-Write" Anomaly Explained

When an API receives a message send request, it must write a row in the `messages` table (so customer service sees the message in chat history) and notify the sender worker. If an external broker like Redis or RabbitMQ is used:

```
// THE BROKEN PATTERN (BullMQ / RabbitMQ):
await db.Message.create({ id: 104, text: "Your order shipped", status: "queued" });
// <-- If server crashes or Redis network times out right here:
await redisQueue.add({ messageId: 104 }); // NEVER RUNS!
// Result: Message 104 is stuck in PostgreSQL as "queued" forever.
```

The PostgreSQL Outbox Solution:

```
// THE ATOMIC SOLUTION (PostgreSQL Outbox):  
await db.sequelize.transaction(async (t) => {  
  const msg = await db.Message.create({ ... }, { transaction: t });  
  await db.OutboxJob.create({ messageId: msg.id, ... }, { transaction: t });  
});  
// Result: Both commit together or both roll back together. 100% ACID consistency.
```

3. Technology Stack Justification & Trade-Off Analysis

1. Runtime & Framework: Node.js 20+ (ESM) + Express 5.0

- **Why not Python (FastAPI) or Go:** The de facto open-source WhatsApp multi-device engine, *Baileys*, is built natively in TypeScript for Node.js. Running the gateway in Python or Go would require an inter-process gRPC/IPC bridge to a Node.js sidecar, introducing serialization latency, socket disconnection hazards, and duplicate memory consumption.
- **Why Express 5.0:** Native Promise rejection handling, modern regex route parameters, and seamless middleware cascading without unhandled rejection crashes.

2. Vector Database: PostgreSQL + pgvector

- **Why not Pinecone / Weaviate / Milvus:** Pinecone introduces recurring per-pod or per-query SaaS fees and requires synchronizing data between PostgreSQL and external cloud endpoints. By leveraging `pgvector` in the existing PostgreSQL database, relational session data and high-dimensional embeddings (768-d) live in the same ACID-compliant engine.
- **Zero-Dependency Cosine Fallback:** The system includes a custom PostgreSQL PL/pgSQL function `cosine_similarity(float8[], float8[])`. If running on standard Windows PostgreSQL installations without C-extension binaries, the vector search continues to operate with mathematical parity.

3. AI Intelligence: Google Gemini 1.5 Flash via LangChain JS

- **Why not OpenAI GPT-4o or Claude 3.5 Sonnet:** Gemini 1.5 Flash provides an industry-leading free tier (15 requests/min, 1,500 requests/day, 1M tokens/min) with ultra-low latency (<600ms Time-to-First-Token). Its 1-million-token context window eliminates context-truncation penalties when passing detailed company manuals.
- **LangChain JS Integration:** Standardizes text chunking (`RecursiveCharacterTextSplitter`), prompt template parameterization, and clean switching between models without rewriting business logic.

4. WhatsApp Driver: Baileys WebSocket Protocol

- **Why not Official WhatsApp Cloud API:** Meta's Cloud API charges per 24-hour conversation window and requires extensive Meta Business Manager verification. Baileys connects via native WhatsApp Web multi-device WebSockets with zero per-message charges.
- **Why not Puppeteer / Selenium:** Headless Chrome automation consumes 800MB–1.5GB RAM per session and frequently crashes when WhatsApp Web updates its DOM. Baileys operates purely in JavaScript memory (<40MB RAM per session) using WhatsApp's binary Protobuf protocol.

4. Multi-Tier Anti-Ban Engine & Hard Rate Limiting

WhatsApp uses automated behavioral heuristics to flag spam bots. The gateway implements an enterprise **Anti-Ban Engine** (`src/services/antiBan.service.js`) enforcing 6 strict mathematical barriers:

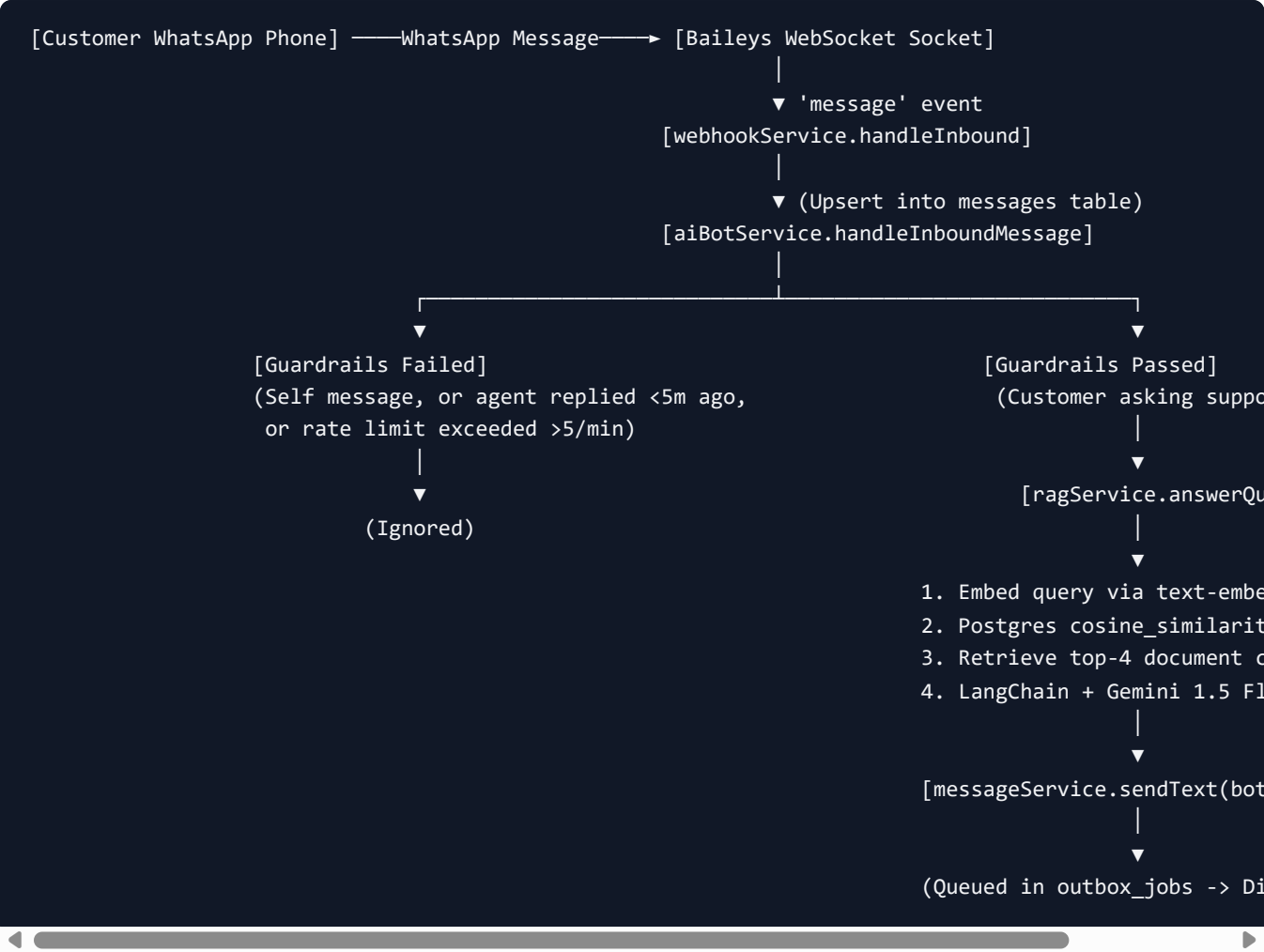
- 1 Rolling 60-Second Window (Max 4 msgs/min in Strict Mode):** Instead of an approximate counter, timestamps of every sent message are kept in a rolling 60-second array. If 4 messages have been sent within the last 60 seconds, the worker calculates the exact millisecond cooldown remaining and reschedules the job.
- 2 Hard Inter-Message Gap (Minimum 5.0 Seconds):** Enforces a mandatory hardware cooldown of 5,000ms between any two consecutive dispatches, preventing burst spikes.
- 3 Same-Recipient Flood Protection (Minimum 10.0 Seconds):** Tracks contact JIDs. If a client application queues multiple rapid messages to the same phone number, the engine inserts a 10-second gap between them.
- 4 Cold Contact Velocity Limiter (Max 6 New Contacts / Hour):** WhatsApp flags accounts that message dozens of brand-new, unsaved contacts. The engine tracks new vs. existing contacts; the 7th new contact in an hour is deferred to the next hour.
- 5 Natural Burst Pauses ("Coffee Break"):** After every 10 to 15 continuous messages, the engine injects a human-like pause of 45 to 90 seconds.
- 6 Daily Warmup Schedule (Day 1: 25 msgs/day):** Fresh numbers start with a strict daily cap of 25 messages, ramping up by 20% daily at midnight up to 300/day. Once the daily limit is hit, jobs sleep in PostgreSQL until 00:05 the next day.

5. End-to-End System Flows

Flow A: Message Sending & Anti-Ban Execution



Flow B: Inbound Customer Message & AI RAG Auto-Reply



6. Complete REST & Webhook API Reference

A. Session Management

Method	Endpoint	Description
POST	/api/v1/sessions	Creates a new WhatsApp session. Body: {"id": "sales-1", "name": "Sales Line"} .
GET	/api/v1/sessions/:id/qr	Renders the multi-device QR pairing code as image (format=png) or JSON (format=json).
DELETE	/api/v1/sessions/:id	Logs out the WhatsApp socket and destroys stored session state.
POST	/api/v1/sessions/:id/simulate/incoming	Injects a mock customer message for sandbox testing with zero ban risk.

B. Message Sending Endpoints

1. Send Plain Text

POST /api/v1/sessions/sales-1/messages/text

Authorization: Bearer <API_KEY>

Content-Type: application/json

```
{
  "to": "919876543210",
  "text": "Hello! Your package has been dispatched via Express Delivery.",
  "idempotencyKey": "order-9912-dispatched"
}
```

2. Upload and Send Voice Note (Microphone Blob or Audio File)

POST /api/v1/sessions/sales-1/messages/voice/upload

Authorization: Bearer <API_KEY>

Content-Type: multipart/form-data

file: [binary .webm / .ogg / .mp3 audio]

to: "919876543210"

idempotencyKey: "voice-note-101"

3. Upload and Send Image

POST /api/v1/sessions/sales-1/messages/image/upload

Authorization: Bearer <API_KEY>

Content-Type: multipart/form-data

file: [binary image: .png / .jpg]

to: "919876543210"

caption: "Product catalog attached"

4. Upload and Send Document / PDF

POST /api/v1/sessions/sales-1/messages/document/upload

Authorization: Bearer <API_KEY>

Content-Type: multipart/form-data

file: [binary document: .pdf / .docx]

to: "919876543210"

C. RAG Knowledge Base Endpoints

```
# 1. Ingest Knowledge Document into Vector Store
POST /api/v1/rag/documents/upload
Authorization: Bearer <API_KEY>
Content-Type: multipart/form-data

file: [binary document: .pdf / .docx / .txt / .md]
title: "Shipping and Return Policy 2026"

# 2. Test RAG Semantic Query
POST /api/v1/rag/query
Authorization: Bearer <API_KEY>
Content-Type: application/json

{
  "query": "How do I return a damaged item?",
  "topK": 3
}

# 3. Update AI Bot & Gemini API Key Settings
POST /api/v1/rag/settings
Authorization: Bearer <API_KEY>
Content-Type: application/json

{
  "apiKey": "AIzaSy...",
  "botEnabled": true,
  "sessionId": "global"
}
```

D. Inbound Webhook Specification & Verification

When an inbound message is received or message status updates (sent, delivered, read), subscribed webhooks receive an HTTP POST with header `X-Signature: sha256=...` computed with your webhook secret.

```
// Receiver-Side HMAC Verification (Node.js):
import crypto from 'crypto';

function verifyWebhook(req, secret) {
  const incomingSignature = req.headers['x-signature'];
  const computedSignature = 'sha256=' + crypto
    .createHmac('sha256', secret)
    .update(JSON.stringify(req.body))
    .digest('hex');

  return crypto.timingSafeEqual(
    Buffer.from(incomingSignature),
    Buffer.from(computedSignature)
  );
}
```